APAS

Adaptive Pedagogy & Assessment System

Technical Architecture Document

Version 1.0 | April 2026

Classification: Internal / Technical

1. Technology Stack Overview

APAS is built as a modern single-page application with a serverless backend. Below is the complete technology stack.

1.1 Frontend

Technology	Version	Purpose
React	18.3	UI library with hooks-based component architecture
TypeScript	5.x	Type-safe development across all source files
Vite	5.x	Build tool and dev server with HMR
Tailwind CSS	3.x	Utility-first CSS framework
React Router DOM	6.x	Client-side routing with nested routes
TanStack React Query	5.x	Server state management, caching, and mutations
Recharts	2.x	Data visualization for analytics dashboards
shadcn/ui + Radix	Latest	Accessible, themeable UI component library
Lucide React	Latest	Icon system
react-hook-form + Zod	Latest	Form handling with schema-based validation
xlsx	0.18	Client-side Excel file parsing for bulk imports
html2pdf.js	0.14	Client-side PDF report generation

1.2 Backend (Serverless)

Technology	Purpose
Supabase (PostgreSQL 15)	Database, authentication, RLS, real-time subscriptions
Supabase Auth	Email/password authentication with JWT tokens
Supabase Edge Functions (Deno)	Serverless API endpoints for AI integrations
Supabase Storage	File storage for textbooks (public bucket)
Row-Level Security (RLS)	Fine-grained data access control per user role

1.3 AI / ML Integrations

Service	Model	Use Case
Lovable AI Gateway	google/gemini-2.5-flash	Curative lesson plan generation (streaming)
Groq	llama-3.3-70b-versatile	MCQ generation, test analysis, period plans
Groq	llama-3.3-70b-versatile (temp 0.3)	Deterministic performance analysis

1.4 DevOps & Tooling

Tool	Purpose
Lovable Cloud	Hosting, deployment, CI/CD pipeline
ESLint 9	Code linting with React hooks plugin
Vitest	Unit and integration testing
PostCSS + Autoprefixer	CSS processing

2. System Architecture

The application follows a three-tier architecture pattern with clear separation of concerns.

2.1 Architecture Diagram

```
+-----------------------------------------------------------------+
|                        CLIENT BROWSER                           |
|  +-------------------------------------------------------------+  |
|  |  React 18 SPA (Vite Build)                                  |  |
|  |  +------------------+  +----------------+  +------------+  |  |
|  |  | React Router DOM |  | TanStack Query |  | Contexts   |  |  |
|  |  | (6 role-based    |  | (server state  |  | Auth, Notif|  |  |
|  |  |  route groups)   |  |  + cache)      |  | Governance |  |  |
|  |  +------------------+  +----------------+  +------------+  |  |
|  |  +------------------+  +----------------+  +------------+  |  |
|  |  | shadcn/ui + Radix|  | Recharts       |  | xlsx       |  |  |
|  |  | (UI components)  |  | (charts)       |  | (imports)  |  |  |
|  |  +------------------+  +----------------+  +------------+  |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
            |                    |                    |
     Supabase JS SDK    REST API (anon key)   Realtime WS
            |                    |                    |
+-----------------------------------------------------------------+
|                      SUPABASE BACKEND                           |
|  +--------------------+  +-------------------+  +-----------+    |
|  | Auth (GoTrueV2)    |  | Edge Functions    |  | Storage   |    |
|  | JWT + email/pass   |  | (Deno runtime)    |  | textbooks |    |
|  +--------------------+  +-------------------+  +-----------+    |
|  +-------------------------------------------------------------+  |
|  |              PostgreSQL 15 + Row-Level Security             |  |
|  |  25+ tables | 80+ RLS policies | triggers | functions      |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
            |                              |
    +----------------+           +------------------+
    | Lovable AI GW  |           | Groq Cloud API   |
    | Gemini 2.5     |           | LLaMA 3.3 70B    |
    +----------------+           +------------------+
```

3. Database Schema

The database consists of 25+ tables organized into functional domains. All tables use UUID primary keys with gen_random_uuid() defaults.

3.1 Core Identity Tables

profiles

Stores user identity linked to auth.users. Auto-created via handle_new_user() trigger.

Column	Type	Nullable	Description
id	UUID (PK)	No	Matches auth.users.id
full_name	TEXT	Yes	Display name
role	TEXT	No	student | teacher | school_admin | admin
created_at	TIMESTAMPTZ	No	Auto-set on creation

students

Extended student data. UNIQUE constraint on profile_id prevents duplicates.

Column	Type	Nullable	Description
id	UUID (PK)	No	Student record ID
profile_id	UUID (FK)	No	References profiles.id (UNIQUE)
grade	TEXT	Yes	Class level (e.g., Class 3)
age	INTEGER	Yes	Student age
roll_number	TEXT	Yes	Class roll number
vark_type	TEXT	Yes	Learning style (V/A/R/K)
dominant_intelligence	TEXT	Yes	MI type from assessment
zpd_score	INTEGER	Yes	Zone of Proximal Development
curriculum	TEXT	Yes	CBSE / ICSE / State Board
parent_phone	TEXT	Yes	Parent contact number
parent_email	TEXT	Yes	Parent email address

3.2 Classroom Management

classes

Column	Type	Description
id	UUID (PK)	Class ID
name	TEXT	Class name (e.g., Class 3)
section	TEXT	Section (default: A)
created_by	UUID (FK)	Creator profile ID

class_students

Junction table mapping students to classes.

Column	Type	Description
class_id	UUID (FK)	References classes.id
student_id	UUID (FK)	References students.id
assigned_by	UUID (FK)	Admin who assigned

class_teachers

Maps teachers to classes with role and subject.

Column	Type	Description
class_id	UUID (FK)	References classes.id
teacher_id	UUID (FK)	References profiles.id
teacher_role	TEXT	primary | assistant
subject	TEXT	Subject taught in this class

3.3 Assessment & Diagnostic Tables

assessments

Stores VARK + MI + ZPD scores from the self-assessment module.

Column	Type	Description
student_id	UUID (FK)	References students.id
visual_score	INTEGER	VARK Visual score
auditory_score	INTEGER	VARK Auditory score
reading_score	INTEGER	VARK Reading score
kinesthetic_score	INTEGER	VARK Kinesthetic score
mi_scores	JSONB	Multiple Intelligence scores object
zpd_math/science/english	INTEGER	ZPD per subject

student_assessments

Teacher-submitted student observation assessments with class/section/curriculum metadata.

Column	Type	Description
student_name	TEXT	Student display name
student_age	INTEGER	Age at assessment
age_group	INTEGER	Mapped group (5, 10, or 15)
teacher_id	UUID (FK)	Assessing teacher
responses	JSONB	Question-answer map
student_class	TEXT	Class at assessment time
section	TEXT	Section at assessment time
curriculum	TEXT	Curriculum at assessment time

teacher_assessments

Teacher observation assessment linked to student profile.

Column	Type	Description
teacher_id	UUID (FK)	Teacher who assessed
student_profile_id	UUID (FK)	Student profile reference
student_name	TEXT	Student name
age_group	INTEGER	Assessment age group
responses	JSONB	Observation responses

diagnostic_requests

Governs the diagnostic test approval workflow (pending -> approved -> assigned -> completed).

Column	Type	Description
teacher_id	UUID (FK)	Requesting teacher
class_name	TEXT	Target class
section	TEXT	Target section
subject	TEXT	Subject area
purpose	TEXT	Reason for diagnostic
suggested_count	INTEGER	Requested question count
status	TEXT	pending|approved|rejected|assigned|completed
approved_by	UUID (FK)	Approving admin
approved_count	INTEGER	Approved question count
question_distribution	JSONB	Per-category distribution
questions	JSONB	Generated question set

diagnostic_submissions

Student answers to assigned diagnostic tests.

Column	Type	Description
request_id	UUID (FK)	References diagnostic_requests.id
student_id	UUID	Student auth user ID
answers	JSONB	Answer data
score	INTEGER	Total score
total_questions	INTEGER	Question count

3.4 Lesson & Curriculum Tables

lessons

AI-generated and manually created lesson plans.

Column	Type	Description
title	TEXT	Lesson title (Class X-Section Subject Topic)
subject	TEXT	Subject area
topic	TEXT	Specific topic
class_level	TEXT	Target class
section	TEXT	Target section
curriculum	TEXT	Curriculum standard
vark_target	TEXT	Target learning style
approach	TEXT	Teaching approach
delivery_method	TEXT	Delivery mechanism
duration_minutes	INTEGER	Lesson duration
lesson_content	TEXT	Full markdown content
learning_outcomes	TEXT	Expected outcomes
content	JSONB	Structured content data
ai_generated	BOOLEAN	Whether AI created this

lesson_assignments

Tracks which lessons are assigned to which students.

Column	Type	Description
lesson_id	UUID (FK)	References lessons.id
student_id	UUID (FK)	References students.id
assigned_by	UUID (FK)	Teacher who assigned
status	TEXT	assigned | completed

period_plans

Period-wise teaching plans generated from lessons.

Column	Type	Description
lesson_id	UUID (FK)	Source lesson
teacher_id	UUID	Owning teacher
class_level / section	TEXT	Target class

subject	TEXT	Subject
plan_data	JSONB	Period-by-period breakdown
total_teaching_days	INTEGER	Default 20
period_duration	INTEGER	Minutes per period (default 40)
periods_per_week	INTEGER	Default 5
is_locked	BOOLEAN	Prevents further edits

3.5 Homework Tables

homework_assignments

Column	Type	Description
lesson_id	UUID (FK)	Source lesson
assigned_by	UUID	Teacher profile ID
title	TEXT	Assignment title
class_level / section	TEXT	Target class
subject	TEXT	Subject
questions	JSONB	Question array
status	TEXT	active | closed

homework_submissions

Column	Type	Description
assignment_id	UUID (FK)	References homework_assignments.id
student_id	UUID	Student auth user ID
answers	JSONB	Student answers
score	NUMERIC	Auto-calculated score

3.6 Performance & Analytics

performance_records

Column	Type	Description
student_id	UUID (FK)	References students.id
lesson_id	UUID (FK)	References lessons.id
pretest_score	INTEGER	Score before lesson
posttest_score	INTEGER	Score after lesson
normalized_gain	NUMERIC	Calculated learning gain
mastery_score	INTEGER	Overall mastery level
effort_score	INTEGER	Student effort rating

academic_tests

AI-generated MCQ tests taken by students.

Column	Type	Description
student_id	UUID	Student auth user ID
subject	TEXT	Test subject
student_class	TEXT	Class level
section	TEXT	Section
questions	JSONB	Generated MCQ array
answers	JSONB	Student answer map
score	INTEGER	Points scored
total_questions	INTEGER	Total question count

mismatch_alerts

System-generated alerts when teaching approach mismatches student needs.

Column	Type	Description
student_group	TEXT	Affected student cohort
lesson_type	TEXT	Current lesson approach
fail_rate	INTEGER	Failure percentage
trigger_condition	TEXT	What triggered the alert
recommendation	TEXT	Suggested corrective action
status	TEXT	flagged | resolved

3.7 Gamification Tables

student_game_profiles

Column	Type	Description
student_id	UUID	Student auth user ID
brain_level	TEXT	reactive | adaptive | strategic | creative
brain_xp	INTEGER	Experience points for brain level
age_stage	TEXT	explorer | challenger | etc.
student_age	INTEGER	Age for difficulty calibration
total_games_played	INTEGER	Lifetime game count
avg_accuracy	NUMERIC	Running accuracy average
daily_goal	INTEGER	Games per day target
games_today	INTEGER	Games played today
surprise_reward_available	BOOLEAN	Bonus reward flag

game_sessions

Column	Type	Description
student_id	UUID	Player ID
game_type	TEXT	Game identifier
game_category	TEXT	Category (math, language, etc.)
subject	TEXT	Academic subject mapping
score / max_score	INTEGER	Points earned vs possible
accuracy	NUMERIC	Percentage correct
difficulty_reached	INTEGER	Max difficulty tier
avg_response_time	INTEGER	Milliseconds per question
time_used	INTEGER	Total seconds played

user_gamification / xp_transactions / achievement_definitions / user_achievements

These four tables handle the XP economy: user_gamification tracks levels and streaks, xp_transactions logs every XP event, achievement_definitions defines unlockable badges, and user_achievements records earned badges.

3.8 Governance & Notifications

governance_notifications

Column	Type	Description
user_id	UUID	Recipient
event_type	TEXT	e.g., diagnostic_request_submitted
title	TEXT	Notification title
message	TEXT	Full message body
reference_id	UUID	Related entity ID
reference_type	TEXT	Entity type (diagnostic_request)
channel	TEXT	in_app (default)
is_read	BOOLEAN	Read status

teacher_question_assignments

Maps which assessment questions are assigned to which teachers per age group.

4. Entity Relationship Overview

```
auth.users (Supabase managed)
     |
     | 1:1 (trigger: handle_new_user)
     v
profiles (id, full_name, role)
     |
     +--- 1:1 ---> students (profile_id UNIQUE)
     |                  |
     |                  +--- 1:N ---> assessments
     |                  +--- 1:N ---> performance_records
     |                  +--- N:M ---> classes (via class_students)
     |                  +--- 1:N ---> lesson_assignments
     |
     +--- N:M ---> classes (via class_teachers)
     |
     +--- 1:N ---> diagnostic_requests (teacher_id)
     +--- 1:N ---> student_assessments (teacher_id)
     +--- 1:N ---> teacher_assessments (teacher_id)
     +--- 1:N ---> governance_notifications (user_id)
     +--- 1:1 ---> user_gamification (user_id)

lessons
     +--- 1:N ---> lesson_assignments
     +--- 1:N ---> performance_records
     +--- 1:N ---> period_plans
     +--- 1:N ---> homework_assignments
                        +--- 1:N ---> homework_submissions

diagnostic_requests
     +--- 1:N ---> diagnostic_submissions
```

5. Row-Level Security (RLS) Model

Every table has RLS enabled. Access is controlled via 80+ policies using the get_user_role() security-definer function.

5.1 Access Control Matrix

Table	Student	Teacher	School Admin	Master Admin
profiles	Read own + teachers	Read all students	Read all	Read all
students	Read/Update own	Read all, Insert	Full CRUD	Full CRUD
classes	Read	Read	Full CRUD	Full CRUD
class_students	Read own	Read all	Full CRUD	Full CRUD
class_teachers	-	Read own	Full CRUD	Full CRUD
lessons	Read	Full CRUD	Read	Full CRUD
assessments	Read/Insert own	Read all	Read all	Full CRUD
diagnostic_requests	Read assigned	CRUD own	Read + Update	Full CRUD
diagnostic_submissions	Read/Insert own	Read all	Read all	Full CRUD
performance_records	Read own	Read/Insert/Update	Read	Full CRUD
homework_assignments	Read (class match)	CRUD own	-	-
homework_submissions	CRUD own	Read (own assigns)	-	-
game_sessions	Read/Insert own	Read all	-	-
governance_notifications	Read own	Read own, Insert	Read own, Insert	Read own, Insert

5.2 Security-Definer Function

The get_user_role() function runs with SECURITY DEFINER privileges to avoid recursive RLS checks when policies reference the profiles table.

```
CREATE FUNCTION public.get_user_role(_user_id UUID)
RETURNS TEXT
LANGUAGE SQL STABLE SECURITY DEFINER
SET search_path = public
AS $$ SELECT role FROM public.profiles WHERE id = _user_id; $$
```

6. Database Functions & Triggers

6.1 handle_new_user()

Trigger function on auth.users INSERT. Auto-creates a profiles row and, for student role, a students row with grade and age from user metadata.

6.2 notify_diagnostic_request_submitted()

Trigger on diagnostic_requests INSERT. Sends governance_notifications to all admin and school_admin users when a teacher submits a new diagnostic request.

6.3 notify_diagnostic_request_decided()

Trigger on diagnostic_requests UPDATE. Fires notifications when status transitions: pending->approved/rejected (notifies teacher), any->assigned (notifies admins), any->completed (notifies teacher + admins).

7. Edge Functions (Serverless API)

All edge functions run on the Deno runtime and are deployed automatically via Lovable Cloud.

Function	JWT Verify	AI Model	Purpose
curative-assistant	false	Gemini 2.5 Flash (Lovable AI GW)	Streams curative lesson plans with textbook context injection
generate-lessons	false	Gemini 2.5 Flash	Legacy lesson generation
generate-mcqs	default	LLaMA 3.3 70B (Groq)	Generates subject-specific MCQ tests
analyze-test-results	default	LLaMA 3.3 70B (Groq, temp 0.3)	Analyzes test performance with strengths/weaknesses
generate-period-plans	default	LLaMA 3.3 70B (Groq)	Creates period-wise teaching schedules
create-students-batch	default	None	Bulk creates auth users + student records
list-subjects	default	None	Returns subject list from textbook storage

7.1 Edge Function Flow: Curative Lesson Generation

```
Teacher UI                    Edge Function              AI Gateway
    |                             |                         |
    |-- POST /curative-assistant ->|                         |
    |   {class, subject, topic,   |                         |
    |    curriculum, varkProfile}  |                         |
    |                             |-- List textbook files -->|
    |                             |<-- File list ------------|
    |                             |                         |
    |                             |-- Match textbook by    |
    |                             |   subject + class      |
    |                             |                         |
    |                             |-- Stream prompt ------->|
    |                             |   (system + textbook   |
    |                             |    context + user msg)  |
    |                             |                         |
    |<-- SSE stream (markdown) ---|<-- Token stream --------|
    |                             |                         |
    |-- Save to lessons table --->|                         |
```

8. Authentication Architecture

8.1 Auth Flow

- Email/password authentication via Supabase Auth (GoTrue V2)
- JWT tokens stored in localStorage with auto-refresh
- handle_new_user() trigger auto-creates profile + student record on signup
- Role is set via user_metadata during registration
- Email confirmation is required (auto-confirm disabled)

8.2 Session Management

```
AuthContext (React Context)
  |-- onAuthStateChange() listener
  |-- getSession() on mount
  |-- fetchProfile() after auth
  |-- Provides: session, user, profile, loading, signOut

ProtectedRoute (wrapper component)
  |-- Checks loading state
  |-- Redirects to /login if no session

RoleGuard (wrapper component)
  |-- Checks profile.role against allowedRoles[]
  |-- Shows access denied if unauthorized
```

8.3 Role Hierarchy & Route Protection

Route	Allowed Roles
/dashboard	All authenticated users
/diagnostic	All authenticated users
/curative	teacher, admin
/analytics	teacher, admin, school_admin
/alerts	admin only
/teacher	teacher, admin, school_admin
/gamification	student, admin
/academic-tests	student only
/admin	admin, school_admin
/requests	teacher only
/settings	All authenticated users

9. Storage Architecture

9.1 Textbook Storage

- Bucket: textbooks (public)
- Purpose: Stores curriculum textbook files for AI context injection
- Access: Public read for all; upload managed by admins
- File naming convention: Subject_ClassLevel.pdf (matched by edge functions)

9.2 Client-Side File Processing

- Excel imports: Parsed client-side using xlsx library for bulk student creation
- PDF reports: Generated client-side using html2pdf.js for student/class reports

10. API Keys & Secrets

All sensitive keys are stored as Supabase secrets (encrypted at rest) and accessed only from Edge Functions.

Secret Name	Used By	Purpose
LOVABLE_API_KEY	curative-assistant, generate-lessons	Lovable AI Gateway authentication
GROK_API_KEY	generate-mcqs, analyze-test-results, generate-period-plans	Groq API authentication
SUPABASE_SERVICE_ROLE_KEY	create-students-batch	Admin-level DB operations (bypass RLS)
SUPABASE_URL	All edge functions	Database connection endpoint
SUPABASE_ANON_KEY	Edge functions (client init)	Public API key for Supabase client

11. Key Data Flows

11.1 Student Onboarding Flow

```
Admin/Teacher                    System                    Database
    |                              |                          |
    |-- Upload Excel / Manual -->  |                          |
    |                              |-- Edge: create-students  |
    |                              |   -batch (service role) -|->
    |                              |                          |
    |                              |   auth.admin.createUser()|
    |                              |   -> trigger:            |
    |                              |      handle_new_user()   |
    |                              |      -> profiles INSERT  |
    |                              |      -> students INSERT  |
    |                              |                          |
    |                              |-- Update student with -->|
    |                              |   roll_number, phone,    |
    |                              |   email, grade           |
    |<-- Results (success/fail) ---|                          |
```

11.2 Diagnostic Governance Flow

```
Teacher          School Admin           System              Students
   |                  |                    |                    |
   |-- Submit req --> |                    |                    |
   |                  |                    |-- trigger:         |
   |                  |                    |   notify_submitted |
   |                  |<-- Notification ---|                    |
   |                  |                    |                    |
   |                  |-- Approve/Reject ->|                    |
   |                  |                    |-- trigger:         |
   |<-- Notification -|                    |   notify_decided   |
   |                  |                    |                    |
   |-- Assign ques -->|                    |                    |
   |                  |                    |-- Map class to     |
   |                  |                    |   age group,       |
   |                  |                    |   build question   |
   |                  |                    |   set              |
   |                  |                    |                    |
   |                  |                    |-- Status: assigned |
   |                  |                    |                    |
   |                  |                    |         Questions->|
   |                  |                    |         <-Answers  |
   |                  |                    |-- Status:completed |
```

11.3 Normalized Gain Calculation

```
Formula: g = (posttest - pretest) / (100 - pretest)

If pretest = 100: gain = 1.0 (already mastered)
If posttest < pretest: gain = negative (regression)
```

```
Interpretation:
  g >= 0.7  : High gain (excellent learning)
  0.3 <= g < 0.7 : Medium gain (satisfactory)
  g < 0.3   : Low gain (needs intervention)
```

12. Frontend Architecture

12.1 Directory Structure

```
src/
  components/
    layout/          AppHeader, AppSidebar, AppLayout
    gamification/    Game components + engine/
    report/          ReportContent, reportHtmlGenerator
    ui/              shadcn components (40+ components)
  contexts/          AuthContext, NotificationContext, GovernanceNotificationContext
  data/              Static data (assessment questions, VARK mapping)
  hooks/             use-mobile, use-toast, useGamification
  integrations/      supabase/ (auto-generated client + types)
  pages/             12 route pages
```

12.2 State Management Strategy

State Type	Technology	Example
Server state	TanStack React Query	Student lists, lessons, assessments
Auth state	React Context (AuthContext)	Session, user, profile, loading
UI state	React useState/useReducer	Modal open/close, form inputs
Notifications	React Context	Governance + general notifications
URL state	React Router	Current route, navigation

12.3 Component Architecture Pattern

```
Page Component (e.g., Curative.tsx)
  |-- Uses AppLayout for sidebar + header
  |-- Contains page-specific state and logic
  |-- Calls supabase SDK directly or via hooks
  |-- Renders domain components:
       |-- CurativeLessonPlanView (AI streaming)
       |-- PeriodPlanGenerator (period-wise breakdown)
       |-- LessonPerformanceSummary (analytics)
       |-- BulkScoreEntryModal (performance input)
```

--- End of Document ---